Mintfy

Software Requirements Specification

Module	SE3092 Platform Based Development, Assignment 01 (Year 3, Semester 2, 2026)
Project	Mintfy (package: com.mintfy.app)
Persona	Kavindu Silva, 25, junior software engineer, Colombo
Document status	Final
Date	20 October 2026
Team members	W.P.S.T. Weerawardhana (IT23596870) Tharuka HMND (IT23689176) A.A.R.A.I. Perera (IT23746350)

1. Introduction

1.1 Purpose

This document specifies the requirements for Mintfy, a personal finance app for Android. The requirements are derived from the scenario of Kavindu Silva, a junior software engineer in Colombo who earns from four different sources but cannot explain where his money goes. The document is written for the lecture panel and for our team, as the basis for the Firestore schema, the MVVM architecture, the wireframes and the implementation. It moves from an analysis of the scenario, through an evidence table, functional and non-functional requirements and use cases, to a traceability matrix that links each requirement back to the scenario and forward to the live demo.

1.2 Scope

Mintfy is an Android app, built in Kotlin with Jetpack Compose and Firebase, for people with irregular, multi-source income and a specific savings goal. It lets the user record income from several sources and currencies, record expenses made in cash, by card or from other accounts, see committed and discretionary spending separately, and track progress toward a goal such as Kavindu's LKR 490,000 MacBook. It keeps working offline and syncs when a connection returns. Mintfy cannot import bank data automatically, so every entry is made by the user. Automatic bank or Binance import, live crypto prices, and budgets or spending limits are out of scope (O-01 to O-03), and Google sign-in is deferred (FR-A4). The core problem Mintfy addresses is that Kavindu cannot explain where his money goes and has no view of his progress toward the goal.

1.3 Persona summary: Kavindu Silva

Facts taken from the assignment scenario.

Role	Junior software engineer at a product-focused startup, Colombo; graduated 12 months ago
Salary	LKR 120,000 to 140,000 per month (base plus performance component), paid on the 25th
Freelance	React and WordPress builds for local SMEs, LKR 20,000 to 90,000 per project, milestone-based
AdSense	Tech blog revenue paid monthly in USD, converted to LKR at transfer; roughly LKR 3,500 to 19,000 per month
Crypto	Trades USDT, ETH and altcoins on Binance; gain in Q1, loss in Q3, no running net position
Savings	LKR 11,200 after 12 months and about LKR 1.6 million earned
Goal	MacBook Pro M4, about LKR 490,000, within 12 months
Fixed cost	Rent LKR 34,000; utilities fluctuate
Personality	Technically sharp, impulsive with discretionary spending, optimistic about income, avoidant about expenses
Core frustration	Cannot explain where his money goes; every tracking attempt failed within two weeks

1.4 Definitions

Terms are used with these meanings throughout the document.

Term	Definition
LKR	Sri Lankan rupee. All totals in Mintfy are shown in LKR.
Income source	A named origin of money, of type salary, freelance, AdSense, crypto or other (FR-B1).
Receipt	A record of money actually received from an income source, with a date, amount, currency and LKR value.
Received income	Receipts with the status received. Only received income counts in monthly totals and savings.
Expected income	Money Kavindu anticipates but has not yet received, such as an invoiced freelance milestone. It is never counted as income or savings.
Outstanding invoice	A freelance entry whose status is expected or invoiced, not yet received (FR-B4, FR-B7).
Exchange rate	The number of LKR per one unit of a foreign currency, entered by the user at the time of transfer (A-02).
Expense	Money spent, recorded with an amount, date, category and payment method.
Payment method	How an expense was paid: cash, card, or a named account such as a secondary account (FR-C2).
Category	A label that groups expenses and carries a spending type (FR-D2).
Committed spending	Spending on regular or fixed obligations that are hard to change in the short term, such as rent, utilities, the gym and subscriptions.
Discretionary spending	Optional spending that Kavindu can choose to reduce, such as coffee, dining out and entertainment.
Recurring expense	An expense that repeats at a set frequency, such as rent or a subscription (FR-C6).
Monthly savings	Received income minus recorded expenses for the month (A-01).
Required monthly saving	The remaining goal amount divided by the months left until the deadline (FR-E2).
On track, behind	On track means actual monthly savings are at least the required monthly saving. Behind means they are lower (FR-E4).
Backfilling	Entering a transaction with a past date that was missed at the time (FR-C4).
Interaction	One tap, one list selection or one completed text entry. Used to measure NFR-01.
Sync	Copying changes between the device and Cloud Firestore, so that both hold the same data (FR-G3).
Offline	The device has no network connection. Mintfy must keep working (NFR-03).

2. Scenario Analysis

One subsection for each of the five analytical lenses in section 4.1 of the assignment. For each, the scenario evidence is listed, followed by the analysis and the requirements it produces.

2.1 Income structure

What must the system understand about income structure to show what Kavindu actually earns each month, rather than what he estimates?

Scenario evidence

- Salary arrives on the 25th but varies between LKR 120,000 and 140,000 with bonuses; he rarely reads his payslip.
- Freelance payments are milestone-based and irregular; he forgot to follow up two invoices and once could not match a payment to a project.
- AdSense is paid in USD and converted to LKR at the rate on transfer day; he never records what arrives.
- Crypto trading produced a gain in Q1 and a loss in Q3, and he cannot state his year-to-date position.
- His own estimate of monthly income is 'somewhere between LKR 160,000 and 210,000', never validated.

Analysis

Kavindu earns from four sources, each with a different currency, payment cycle and level of certainty. His salary is the most stable, but it ranges from LKR 120,000 to 140,000 depending on bonuses. AdSense varies from about LKR 3,500 to 19,000 a month and is paid in USD, then converted to LKR at the exchange rate on the transfer day. Freelance income is irregular and milestone-based, and crypto results can be gains or losses. His own estimate of LKR 160,000 to 210,000 a month looks unrealistic: even in the best case, salary plus AdSense reaches only LKR 159,000, so the lower estimate depends on freelance or crypto, the two least predictable sources. His earnings of about LKR 1.6 million over 12 months also average roughly LKR 133,000 a month, below that lower estimate. Mintfy therefore counts income only once it is received and shows expected and received amounts separately, especially for freelance invoices. It records AdSense with the USD amount, the exchange rate and the LKR actually received, and it treats crypto as a net gain or loss rather than regular income.

Requirements this produces

FR-B1 to FR-B9, FR-F1, FR-F6, NFR-05.

2.2 Why four tools failed

What do the four failed tools share behaviourally and in UX, and which Mintfy design decisions prevent the same outcome?

Scenario evidence

- Google Sheet: formulas broke when rows were inserted, data entry took more effort than he would sustain, abandoned after 13 days.
- Play Store budgeting app: USD only, no cloud sync, assumed a single income source and a fixed monthly budget; uninstalled in nine days.
- Notes app with pasted bank exports: unreadable block of unlabelled figures after about four days.
- Bank spending summary: card transactions only, no cash or secondary-account activity, and a PickMe fare labelled online retail.

Analysis

The three tools Kavindu built or installed lasted only 4 to 13 days, and his bank's summary never gave him a full picture. The Google Sheet and the notes file demanded too much manual effort, and the Sheet's formulas broke when he inserted rows. The Play Store app had a rigid data model: it supported only USD, had no cloud sync, and assumed a single income source and a fixed monthly budget. His bank's summary left out cash and secondary-account transactions and mislabelled a PickMe fare as online retail. Finally, gaps in his records made him feel he had failed, which discouraged him from returning. Mintfy therefore

provides quick entry with sensible defaults, supports multiple income sources, currencies and payment methods, and lets him enter missed transactions later without penalty. Architecturally, Firestore provides cloud sync, Room keeps the app working offline, and calculations run in code rather than in formulas he must maintain.

Requirements this produces

FR-B1, FR-B5, FR-C1 to FR-C5, FR-G1 to FR-G3, NFR-01.

2.3 Making the goal real

How should the app connect daily behaviour to the MacBook goal so progress feels real and financially grounded rather than abstract?

Scenario evidence

- Target is about LKR 490,000 within 12 months; current savings are LKR 11,200.
- He does not know his current monthly savings rate, so he cannot know the rate required.
- The 'MacBook Fund' sheet felt discouraging when he returned to incomplete entries.

Analysis

Kavindu wants to save LKR 490,000 but has only LKR 11,200, leaving LKR 478,800. Over 12 months he must save about LKR 39,900 a month, roughly 30% of his average monthly income of about LKR 133,000. This is a large jump from his current savings of only about LKR 933 a month, so he needs to save around 43 times as much. In one month he spent LKR 24,000 on coffee and dining, about 60% of the amount he now needs to save each month, which shows how discretionary spending affects the goal. Mintfy should therefore show the required pace beside his actual savings, say clearly whether he is on track or behind, and connect everyday spending to the goal. The required monthly amount recalculates as time passes and savings build up, so one weak month does not make the goal feel lost. Only income that has actually been received counts toward savings, never expected payments.

Requirements this produces

FR-E1 to FR-E6, FR-F3, FR-F4.

2.4 Expense entry without bank import

How should expense entry minimise friction and maximise long-term adherence, and what must an expense record capture to be useful at month end?

Scenario evidence

- Spending is spread across bank card, cash withdrawals, PickMe, UberEats, a gym auto-debited from a secondary account, and two subscriptions he meant to cancel.
- Rent is LKR 34,000 while utilities fluctuate.
- Reviewing three months of statements, he found LKR 24,000 spent on coffee and casual dining in a single month.
- He has a low tolerance for friction in daily data entry.

Analysis

Because Mintfy cannot import bank data, every expense must be entered by Kavindu himself, and he abandoned earlier tools because the effort of entering data was too high. Mintfy therefore lets him record an expense in no more than three interactions after opening the screen (a tap, a list selection or a completed text entry), with the date, category and payment method defaulting to today and the last ones used. Each field has a purpose: the amount for totals, the date for monthly reports and backfilling, the category to analyse spending (and it also carries the committed or discretionary label, so no extra step is

needed), the payment method to capture cash and secondary-account spending that his bank misses, and an optional note for later recall. Cash purchases can only be recorded by him at the time, so the entry screen has to be fast enough to use at a shop or restaurant. At month end, these fields let Mintfy answer "where did it go" by category, spending type and payment method.

Requirements this produces

FR-C1 to FR-C6, FR-D1 to FR-D3, NFR-01.

2.5 Self-awareness views

Which visualisations reveal the discretionary-to-committed ratio to someone who does not think in categories, and what is the minimum set that gives self-awareness in the first week?

Scenario evidence

- He has no category system and no concept of discretionary vs committed spending.
- He has never seen a visual representation of how his spending is distributed, and is certain it would be clarifying.

Analysis

Kavindu does not think in spending categories, so Mintfy shows his finances in simple terms. The first view is a two-segment donut of committed and discretionary spending, which needs no financial vocabulary because each default category is already tagged. The dashboard offers four essential views: a monthly summary of income, expenses and savings; the donut; a breakdown of top categories that highlights the largest discretionary one; and a goal card showing his progress. Together they show where his money goes and which discretionary spending to look at first. Insight must also appear in the first week rather than after a full month: if he enters rent, the gym and subscriptions on the first day, the committed side fills immediately, and daily entries build the discretionary side as the month goes on.

Requirements this produces

FR-C1, FR-D1, FR-D2, FR-F1 to FR-F4.

3. Evidence-to-Requirement Table

Each scenario fact, the problem it reveals, and the requirement that answers it. This table is what shows the panel your requirements were derived rather than listed.

ID	Scenario evidence	Problem revealed	Requirement(s)
E-01	Salary arrives on the 25th, LKR 120,000 to 140,000, bonus-dependent; rarely checks payslip.	Expected and actual pay differ and he never compares them.	FR-B1, FR-B3
E-02	Freelance pays LKR 20,000 to 90,000 per milestone; twice forgot invoices; once could not match a payment to a project.	Payments are not tied to projects, and unpaid invoices are invisible.	FR-B4, FR-B7, FR-E6
E-03	AdSense paid in USD, converted at transfer; LKR 3,500 to 19,000; never recorded.	The LKR received differs from the USD earned, and nothing records either.	FR-B5
E-04	Crypto: gain in Q1, loss in Q3; no running net position.	Gains and losses are never netted, so year-to-date is unknown.	FR-B9
E-05	Estimates income at LKR 160,000 to 210,000 but never validated.	Monthly income is only ever a guess.	FR-B6, FR-B8, FR-F1, FR-F6
E-06	Spending spread across card, cash, PickMe, UberEats, gym on a secondary account, forgotten subscriptions.	No single record across payment channels; recurring costs go unnoticed.	FR-C1, FR-C2, FR-C6
E-07	Rent LKR 34,000, utilities fluctuate; LKR 24,000 on coffee and dining in one month.	Committed costs vary, and discretionary spending leaks unseen.	FR-D2, FR-D3, FR-F3, FR-C6
E-08	No category system; has never seen how spending is distributed.	He has no model of categories or of committed vs discretionary spending.	FR-D1, FR-D2, FR-F2
E-09	Goal LKR 490,000 in 12 months; savings LKR 11,200; current savings rate unknown.	He cannot compare the required saving (about LKR 39,900 a month) with his actual saving.	FR-E1 to FR-E5, FR-F4
E-10	Google Sheet abandoned: broken formulas after row insertion, discouraging incomplete entries.	Fragile structure, high entry effort, and gaps feel like failure.	FR-C3, FR-C4, FR-C5, NFR-01
E-11	Play Store app uninstalled in nine days: USD only, no cloud sync, single income source, fixed budget.	The tool's data model did not match his income, currency or devices.	FR-A1, FR-B1, FR-B5, FR-G1 to FR-G3
E-12	Notes app became unreadable after about four days.	Unstructured entries cannot be searched or summarised.	FR-C1, FR-C7
E-13	Bank summary: card only, no cash or secondary account, PickMe labelled online retail.	Missing channels and wrong automatic categories destroy trust.	FR-C2, FR-D4, FR-D5

4. Functional Requirements

Priority: Must, Should or Later. Evidence refers to an E row in Section 3. Screen names the screen where the requirement is visible to Kavindu.

Later means outside this submission unless time allows.

4.1 Area A: Authentication and account

ID	Requirement	Priority	Evidence	Screen
FR-A1	The user can register and sign in with email and password.	Must	E-11; Brief s5	Sign in
FR-A2	The session persists across app restarts until the user signs out.	Must	E-10	Sign in
FR-A3	Each user can read and write only their own data.	Must	Brief s4.3	All screens
FR-A4	The user can also sign in with Google.	Later	Brief s5 (bonus)	Sign in

4.2 Area B: Income sources and receipts

ID	Requirement	Priority	Evidence	Screen
FR-B1	The user can create income sources of type salary, freelance, AdSense, crypto or other, each with a name.	Must	E-01 to E-05	Income
FR-B2	The user can record a receipt against a source with a date, amount and currency.	Must	E-01 to E-05	Add income
FR-B3	A salary receipt stores base and bonus separately and shows the difference from the expected salary.	Must	E-01	Add income
FR-B4	A freelance entry stores client, project, milestone amount, due date and a status of expected, invoiced or received.	Must	E-02	Income
FR-B5	A receipt in a foreign currency stores the original amount, currency, exchange rate and LKR value. The user enters the rate used at transfer.	Must	E-03, E-11	Add income
FR-B6	Monthly income is calculated only from receipts with status received.	Must	E-05	Dashboard
FR-B7	The dashboard shows the total of unpaid freelance invoices separately from received income.	Must	E-02	Dashboard
FR-B8	The user can enter an estimated monthly income range, and the app shows actual monthly income against it.	Must	E-05	Dashboard
FR-B9	Crypto results are recorded as signed gains or losses, with a year-to-date net shown.	Should	E-04	Income

4.3 Area C: Expense capture

ID	Requirement	Priority	Evidence	Screen
FR-C1	The user can record an expense with amount, date, category, payment method and an optional note.	Must	E-06, E-12	Add expense
FR-C2	Payment method covers cash, card and user-named accounts such as a secondary account.	Must	E-06, E-13	Add expense
FR-C3	The add-expense form defaults the date to today and the category and payment method to the last ones used.	Must	E-10	Add expense
FR-C4	The user can choose any past date, so missed transactions can be backfilled.	Must	E-10	Add expense
FR-C5	The user can edit or delete any income or expense entry.	Must	E-10	History
FR-C6	The user can mark an expense as recurring with a frequency, and active recurring costs appear in one list.	Should	E-06, E-07	Recurring
FR-C7	The user can browse past income and expenses and filter by date range and category.	Should	E-12	History

4.4 Area D: Categories and committed vs discretionary

ID	Requirement	Priority	Evidence	Screen
FR-D1	On first run the app provides default expense categories (for example Food and Coffee, Transport, Rent, Utilities, Subscriptions, Entertainment) so no setup is needed.	Must	E-08	Add expense
FR-D2	Every category has a spending type of committed or discretionary.	Must	E-07, E-08	Categories
FR-D3	A committed expense can have a different amount each month, such as utilities.	Must	E-07	Add expense
FR-D4	The user can add, rename and change the spending type of categories.	Should	E-13	Categories
FR-D5	The app suggests the category last used for an expense with the same note.	Later	E-13	Add expense

4.5 Area E: Savings goal

ID	Requirement	Priority	Evidence	Screen
FR-E1	The user can create a savings goal with a name, target amount, deadline and amount already saved.	Must	E-09	Goal
FR-E2	The app calculates the remaining amount and the required monthly saving from the time left.	Must	E-09	Goal
FR-E3	Actual monthly savings are received income minus recorded expenses for the month (see A-01).	Must	E-09	Dashboard

ID	Requirement	Priority	Evidence	Screen
FR-E4	The app compares actual savings with the required pace and shows whether the goal is on track or behind.	Must	E-09	Goal, Dashboard
FR-E5	The required monthly saving is recalculated as time passes and savings accumulate.	Must	E-09	Goal
FR-E6	Income that has not been received is never counted toward savings or the goal.	Must	E-02	Goal

4.6 Area F: Dashboard and insights

ID	Requirement	Priority	Evidence	Screen
FR-F1	The dashboard shows this month's received income, expenses and savings.	Must	E-05, E-09	Dashboard
FR-F2	A donut chart shows committed and discretionary spending for the month.	Must	E-08	Dashboard
FR-F3	The dashboard lists top spending categories and highlights the largest discretionary one.	Must	E-07	Dashboard
FR-F4	A goal progress card shows the amount saved, the required pace and the on-track status.	Must	E-09	Dashboard
FR-F5	Dashboard figures update in real time when data changes, with no manual refresh.	Must	Brief s5	Dashboard
FR-F6	Monthly received income is broken down by source.	Should	E-01 to E-05	Dashboard

4.7 Area G: Offline use and sync

ID	Requirement	Priority	Evidence	Screen
FR-G1	The user can add, edit and view transactions with no network connection.	Must	E-11	All screens
FR-G2	Data is cached locally with Room, and screens load from the cache first.	Must	Brief s5	All screens
FR-G3	Changes made offline sync automatically when connectivity returns.	Must	E-11	All screens
FR-G4	If the same record was changed on two devices, the most recent change wins (see A-03).	Should	E-11	None (data layer)
FR-G5	A small indicator shows when the app is offline or has unsynced changes.	Should	E-11	All screens

4.8 Area H: Error and empty states

ID	Requirement	Priority	Evidence	Screen
FR-H1	Empty screens show a message and a button for the next action instead of a blank page.	Must	Brief s4.3	All screens

ID	Requirement	Priority	Evidence	Screen
FR-H2	On network failure the app shows an error with a retry action, and cached data stays visible.	Must	Brief s4.3; E-11	All screens
FR-H3	Forms validate input (amount above zero, required fields) and show messages beside the field.	Must	Brief s4.3	Add income, Add expense
FR-H4	Sign-in and registration errors are shown in plain language.	Must	Brief s4.3	Sign in
FR-H5	The user must confirm before deleting an entry.	Should	E-10	History

5. Non-Functional Requirements

Each non-functional requirement must be measurable, so a reader can tell whether Mintfy meets it.

ID	Category	Measurable requirement	Evidence	Verified by
NFR-01	Usability and friction	Recording an expense takes no more than 3 interactions after opening the add-expense screen. An interaction is one tap, one list selection or one completed text entry. Defaults from FR-C3 count as zero.	E-10, E-11	Timed walkthrough on a device
NFR-02	Performance	The dashboard renders from the local cache within 2 seconds of opening, and a saved entry appears in the totals within 1 second.	E-10	Manual timing on a mid-range device
NFR-03	Offline and reliability	Adding, editing and viewing work in airplane mode, and no screen blocks on a network call.	E-11	Airplane-mode test of each flow
NFR-04	Security and privacy	Firestore rules let a signed-in user read and write only their own documents. No user data is hard-coded.	Brief s4.3	Security rules tests
NFR-05	Data integrity and currency	Money is stored as integer amounts, never floating point. The original amount, currency and rate are kept with the LKR value so any figure can be audited.	E-03	Unit tests on conversion and totals
NFR-06	Compatibility	The app installs and runs on Android 8.0 (API 26) and later, targeting API 34 or higher.	Brief s5	Test on an API 26 and a current emulator
NFR-07	Maintainability	Code follows UI, ViewModel, Repository, Model. Calculations live outside Composables and have unit tests.	Brief s4.3	Code review and unit tests
NFR-08	Error handling	No unhandled exception or blank screen occurs on network failure, empty data or invalid input.	Brief s4.3	Tests with an empty account, airplane mode and bad input

6. Constraints

Constraints are fixed by the assignment brief and are not design choices. Each cites its source.

ID	Concern	Constraint	Source
C-01	Language	Kotlin 100%, no Java in the application layer, Gradle Kotlin DSL	Brief s4.3, s5
C-02	UI	Jetpack Compose with Material Design 3; no XML layouts	Brief s4.3, s5
C-03	Navigation	Navigation Compose, single-activity architecture, typed arguments preferred	Brief s5
C-04	Architecture	MVVM with strict UI, ViewModel, Repository, Model separation; no business logic in Composables; no data-layer calls from the UI	Brief s4.3, s5

ID	Concern	Constraint	Source
C-05	State	StateFlow or LiveData; no raw mutable state exposed from a ViewModel to the UI	Brief s5
C-06	Concurrency	Kotlin Coroutines; viewModelScope for async work	Brief s5
C-07	Authentication	Firebase Authentication; email and password mandatory; Google Sign-In is bonus credit; no hard-coded user data	Brief s4.3, s5
C-08	Database	Cloud Firestore as primary persistence; real-time listeners on the dashboard; Firestore security rules submitted	Brief s4.3, s5
C-09	Local cache	Room strongly recommended for offline persistence; UI must not block on network unavailability	Brief s5
C-10	Dependency injection	Hilt preferred (or Koin); no manual ViewModel instantiation in Composables	Brief s5
C-11	Platform	Minimum SDK API 26 (Android 8.0), target SDK 34 or higher	Brief s5
C-12	Error handling	No unhandled exceptions, no blank screens on network failure, no crashes on empty datasets	Brief s4.3
C-13	Data source	No automatic bank data import is available	Brief s4.1
C-14	Team and time	Group of at most 3; deadline 30 October 2026; live demo of 15 minutes	Brief cover, s6.4
C-15	Deliverables	Private repo with iterative commit history; technical design PDF of at least 15 pages; APK on live Firebase; README	Brief s6

7. Assumptions and Out of Scope

ID	Assumption or out-of-scope item	Reason
A-01	Monthly savings equal received income minus recorded expenses. The app does not read bank balances.	The brief says bank data cannot be imported, so savings must be computable from what is recorded in the app.
A-02	Exchange rates are entered by the user at the time of transfer. There is no live rate feed.	AdSense is converted at the rate on transfer day (scenario s3.2), and a feed would give a different figure from what actually arrived.
A-03	If the same record is edited on two devices, the last write wins.	Mintfy is a single-user app, so conflicts are rare, and this keeps synchronisation simple.
O-01	No automatic import from the bank or Binance.	The brief states automatic bank import is not available.
O-02	No live crypto prices or portfolio valuation.	Kavindu's gap is net gains and losses, not holdings.
O-03	No budgets or spending limits.	His Play Store app assumed a fixed monthly budget and failed (E-11). Mintfy focuses on visibility, not limits.

8. Use Cases

8.1 Actors

Actor	Role
Kavindu (primary user)	Records income and expenses, sets and tracks the savings goal, and reviews the dashboard.
Firebase Authentication	Secondary system actor. Verifies email and password credentials and provides the signed-in identity (UC-01).
Cloud Firestore	Secondary system actor. Stores each user's data in the cloud and syncs changes to and from the device (UC-13).

8.2 Use case list

Derived from the functional requirements. Every Must requirement in Section 4 is covered by at least one use case. All use cases except UC-01 require a signed-in user.

ID	Use case	Actor	Priority	Related FR
UC-01	Register and sign in	Kavindu; Firebase Authentication	Must	FR-A1 to FR-A3, FR-H4
UC-02	Record income receipt	Kavindu	Must	FR-B1 to FR-B3, FR-B5, FR-B6, FR-B9, FR-H3
UC-03	Track freelance invoice	Kavindu	Must	FR-B4, FR-B7
UC-04	Record expense	Kavindu	Must	FR-C1 to FR-C4, FR-D1 to FR-D3, FR-H3
UC-05	Edit or delete an entry	Kavindu	Must	FR-C5, FR-H5
UC-06	View dashboard	Kavindu	Must	FR-B6 to FR-B8, FR-F1 to FR-F3, FR-F5, FR-F6, FR-H1
UC-07	Manage savings goal	Kavindu	Must	FR-E1, FR-E2, FR-E5, FR-H1
UC-08	Check goal progress	Kavindu	Must	FR-E3, FR-E4, FR-E6, FR-F4, FR-H2
UC-09	Set income estimate	Kavindu	Must	FR-B8
UC-10	Manage categories	Kavindu	Should	FR-D2, FR-D4
UC-11	Manage recurring expenses	Kavindu	Should	FR-C6
UC-12	Browse and filter history	Kavindu	Should	FR-C7
UC-13	Sync data	Cloud Firestore	Must	FR-G1 to FR-G5, FR-H2

8.3 Use case diagram

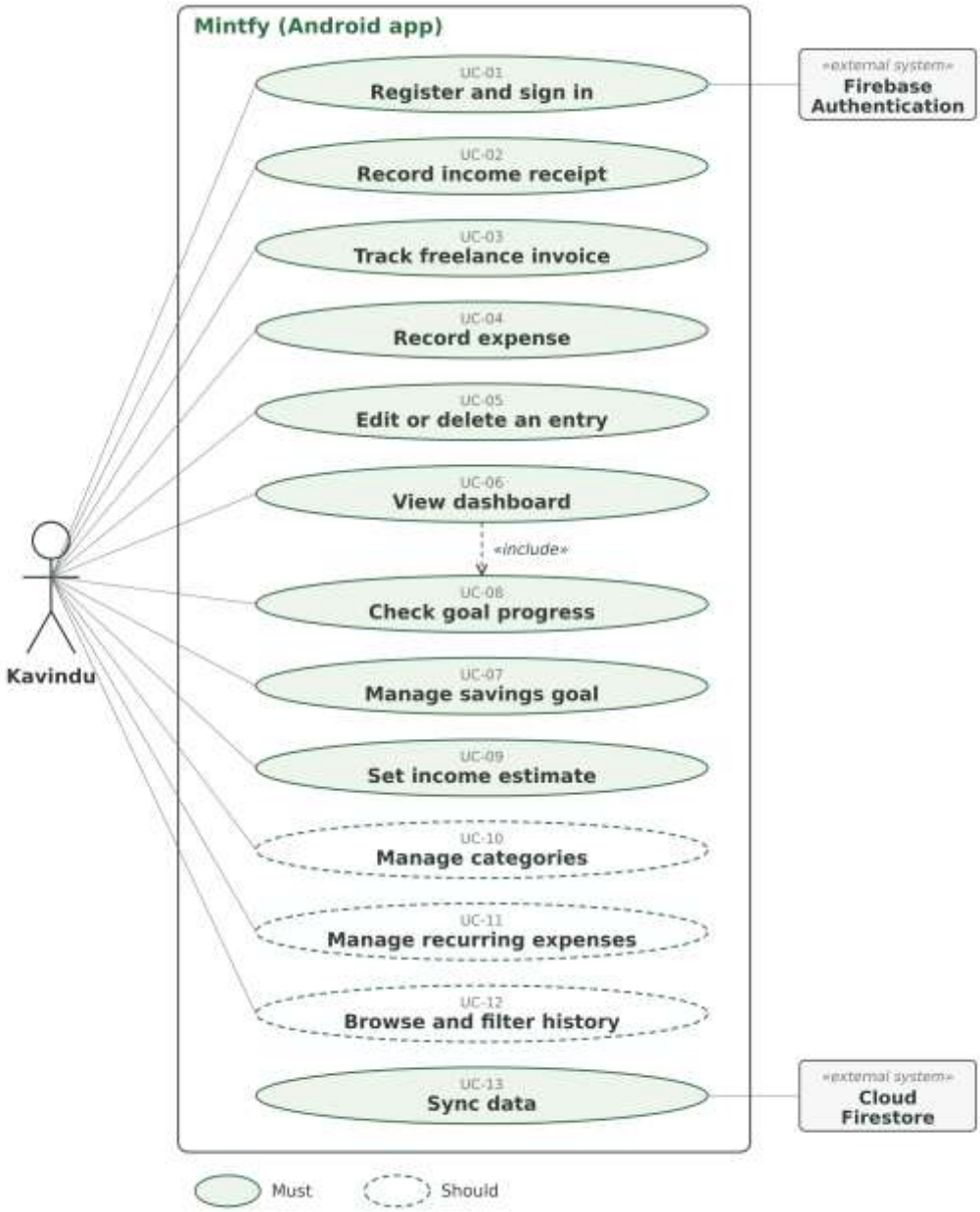


Figure 8.1: Mintfy use case diagram. Solid outline: Must. Dashed outline: Should. Sync (UC-13) is triggered by connectivity, so it is linked to Cloud Firestore only.

8.4 Use case specifications

Three specifications for the use cases that matter most to Kavindu's daily experience: recording an expense (Lens 4), recording income (Lens 1) and checking goal progress (Lens 3).

UC-04 Record expense

Actor	Kavindu
Goal	Record a spending transaction quickly, wherever he is, so that it counts in his monthly picture.
Preconditions	Kavindu is signed in. Default categories exist (FR-D1). A network connection is not required.
Main flow	1. Kavindu opens the add-expense screen. The date shows today, and the category and payment method show the last ones used (FR-C3). 2. He enters the amount. 3. If the category or payment method is different, he selects the correct one. 4. He optionally adds a note. 5. He taps Save. 6. The system validates the input, stores the expense locally and queues it for sync. 7. The dashboard totals and charts update.
Alternate flows	A1 Past date: at step 1 he picks an earlier date to backfill a missed expense (FR-C4). The entry is counted in that month. A2 Cash or secondary account: at step 3 he selects Cash or a named account as the payment method (FR-C2). A3 Invalid input: if the amount is empty or zero, the system shows a message beside the field and does not save (FR-H3). A4 Offline: the expense is saved locally and synced when connectivity returns (FR-G1, FR-G3), and an offline indicator is shown (FR-G5).
Postconditions	The expense is stored with amount, date, category, payment method and note. Its spending type (committed or discretionary) comes from the category. Monthly totals include it.
Related requirements	FR-C1 to FR-C4, FR-D1 to FR-D3, FR-H3, FR-G1, FR-G3; NFR-01, NFR-03

UC-02 Record income receipt

Actor	Kavindu
Goal	Record money he has actually received from one of his sources, so that monthly income reflects reality rather than his estimate.
Preconditions	Kavindu is signed in. At least one income source exists, or he creates one (FR-B1).
Main flow	1. Kavindu opens the add-income screen and selects a source: salary, freelance, AdSense, crypto or other. 2. He enters the date and the amount received. 3. The system shows the fields that match the source type. 4. He taps Save. 5. The system validates the input and stores the receipt with the status received. 6. Monthly income, the breakdown by source and the dashboard update.

Alternate flows	A1 Foreign currency (AdSense): he chooses USD, then enters the USD amount, the exchange rate used at transfer and the LKR received. The system stores all three (FR-B5). A2 Salary: he enters base pay and bonus separately, and the system shows the difference from the expected salary (FR-B3). A3 Crypto: he enters a gain or a loss as a signed amount, and the year-to-date net updates (FR-B9). A4 Invalid input: the system shows a message beside the field and does not save (FR-H3).
Postconditions	The receipt is stored with its original amount, currency, rate and LKR value. It counts toward monthly income because its status is received (FR-B6).
Related requirements	FR-B1 to FR-B3, FR-B5, FR-B6, FR-B9, FR-H3; NFR-05

UC-08 Check goal progress

Actor	Kavindu
Goal	See whether he is on track to buy the MacBook by the deadline.
Preconditions	Kavindu is signed in. A savings goal exists (UC-07).
Main flow	1. Kavindu opens the dashboard or the goal screen. 2. The system calculates the remaining amount: target minus amount saved. 3. The system calculates the required monthly saving from the remaining amount and the months left. 4. The system calculates this month's actual savings: received income minus expenses (A-01). 5. The system compares actual savings with the required pace and shows on track or behind, with both amounts. 6. Unpaid freelance invoices are shown separately and are not counted (FR-E6).
Alternate flows	A1 No goal yet: the system shows an empty state with a button to create one (FR-H1). A2 No data this month: the system shows the required pace, zero actual savings and a short message (FR-H1). A3 Behind: the required pace is recalculated over the remaining months (FR-E5). A4 Network failure: cached figures stay visible, with a retry action (FR-H2).
Postconditions	No data is changed.
Related requirements	FR-E2 to FR-E6, FR-F4, FR-H1, FR-H2

9. Traceability Matrix

Links the scenario evidence through requirements and use cases to the screens, ViewModels and the live demo, so any requirement can be explained from either end. Each row is one use case; its evidence is taken from the requirements it covers (Sections 3 and 4).

Evidence	Requirements	Use case	Screen	ViewModel (planned)	Demo step
E-10, E-11	FR-A1 to FR-A3, FR-H4	UC-01 Register and sign in	Sign in	AuthViewModel	Register a new account, close and reopen the app to show the session persists, then enter a wrong password to show a readable error.
E-01 to E-05, E-11	FR-B1 to FR-B3, FR-B5, FR-B6, FR-B9, FR-H3	UC-02 Record income receipt	Add income	IncomeViewModel	Add an AdSense receipt in USD with its exchange rate and show the stored LKR value. Add a salary with base and bonus.
E-02	FR-B4, FR-B7	UC-03 Track freelance invoice	Income	IncomeViewModel	Create a freelance entry as invoiced, show the unpaid total on the dashboard, then mark it received and show it move into income.
E-06 to E-08, E-10, E-12, E-13	FR-C1 to FR-C4, FR-D1 to FR-D3, FR-H3	UC-04 Record expense	Add expense	ExpenseViewModel	Add an expense using only the defaults (two interactions). Then backfill one dated last week and add one paid in cash.
E-10	FR-C5, FR-H5	UC-05 Edit or delete an entry	History	HistoryViewModel	Edit an amount, then delete an entry after the confirmation, and show the dashboard totals change.
E-01 to E-05, E-07 to E-09	FR-B6 to FR-B8, FR-F1 to FR-F3, FR-F5, FR-F6, FR-H1	UC-06 View dashboard	Dashboard	DashboardViewModel	Show monthly income, expenses and savings, the donut, top categories, and the figures updating live after a new entry.
E-09	FR-E1, FR-E2, FR-E5, FR-H1	UC-07 Manage savings goal	Goal	GoalViewModel	Create the MacBook goal: LKR 490,000, 12 months, LKR 11,200 saved, and show the required LKR 39,900 a month.
E-02, E-09, E-11	FR-E3, FR-E4, FR-E6, FR-F4, FR-H2	UC-08 Check goal progress	Dashboard, Goal	GoalViewModel	Show on track or behind against the required pace. Add an unpaid invoice and show it does not change the savings figure.
E-05	FR-B8	UC-09 Set income estimate	Dashboard	DashboardViewModel	Enter an estimate of LKR 160,000 to 210,000 and show actual monthly income against it.

Evidence	Requirements	Use case	Screen	ViewModel (planned)	Demo step
E-07, E-08, E-13	FR-D2, FR-D4	UC-10 Manage categories	Categories	CategoryViewModel	Change a category's spending type and show the donut change.
E-06, E-07	FR-C6	UC-11 Manage recurring expenses	Recurring	RecurringViewModel	Mark rent and the gym as recurring and show the recurring list.
E-12	FR-C7	UC-12 Browse and filter history	History	HistoryViewModel	Filter the history by date range and by category.
E-11	FR-G1 to FR-G5, FR-H2	UC-13 Sync data	All screens	Data layer (repositories)	Turn on airplane mode, add an expense, show the offline indicator, turn it off, and show the entry arrive in Firestore.

ViewModel names are provisional until the architecture diagram is final. Non-functional requirements NFR-01 to NFR-08 are verified by the methods listed in Section 5, and the demo flow above doubles as the script for the live session.